

How One Coding Agent Learned from Its Own Mistakes

A 120-day case study of capture, retrieval and memory drift

One agent, one operator, one corpus. A hypothesis that fell, and the design that came out of it.

Rogério Faleiros · RFERP LLC · August 2026

Scope, stated first rather than last. This is a case study of a single corpus: one operator, one agent, 120 days. The corpus has commit and documentation discipline well above average, which is exactly why lessons were there to be found. Nothing here is a property of agents in general, and the design in the second half is derived from the case, not validated against an alternative. Review history: three rounds of investigation, one adversarial review of the artefact (23 Aug 2026) whose three blocking findings are corrected in the text, and three rounds of external editorial critique, each of which changed the text.

The problem

I have worked with a coding agent every day for months. In that time it wrote me two very different things and filed them in the same place: where the work stopped, and what we had learned. The first is stale by the next morning. The second should be good for years. When both live in one file, one of two things happens. Either the lesson gets overwritten along with the status, or the file grows until nobody opens it.

The question I asked was simple, and I put it the way a teacher does at the end of class: what did we learn today? The agent already wrote me a handover every evening, so part of the answer existed. But a handover is a notebook of annotations, not a notebook of learning. And I told the agent something that mattered more: learning is not made only of the things that went right. The wrong and the bad are what make you think outside the box.

Three requirements came out of that sentence.

1. Learning is not state. If the text goes stale as the work advances, it is an annotation.
2. The error is raw material, not something to be ashamed of.
3. It has to be found again. A lesson written and never retrieved never happened.

What follows is what we found when we took those three to a real corpus with real numbers. That includes the conclusion that turned out to be wrong, because that is where the useful finding was.

What was measured

The corpus is my own working tree over 120 days: 46 repositories active in the window, 50 session handovers, 167 memory files, 56 per-project instruction files, 2,605 commits. The tree is live, so the window is pinned inside the scripts and re-runs reproduce these numbers. Five recording mechanisms were in use at the same time: handovers, a memory tree, instruction files, curated boards, and append-only decision logs. Everything below is anonymised. No client, project name, company code or master-data name appears.

How the counts were made, so they can be argued with. A *fix commit* is one whose subject line matches `fix|corrig|consert|bug|erro|ajust|repar|resolv`, case-insensitive, in Portuguese or English. That is a generous net: it catches refactors labelled as fixes and misses corrections described in other words, and it is stated here so anyone can re-run it or reject it (*evidencia/02*, script included). *Has a body* means anything beyond the subject line. *Explains itself* means a body of twenty words or more, a threshold set before counting. Third-party clones inside the tree are excluded by an explicit pattern, after a first version of the script counted a vendored repository with 3,730 commits and inflated everything.

Three of the counts below are **not** mechanical, and this matters more than the scripts. "A trap item that describes a real error with its cause", "a memory anchored in an incident" and "a rule with no case behind it" were classified by the agent reading every file, one by one, on 21 Aug 2026. Those are judgements, not measurements. They are reproducible only by someone repeating the reading, and the agent doing the reading is the same agent whose memory is the subject of the study. I flag it here rather than in the limitations, because a reader should know which numbers are countable and which are opinions before seeing them, not after.

Three measurements carry the argument.

The channel that holds the most corrective narrative is the one least likely to be consulted as memory. Of 477 fix commits in the window, 93% have a body and 71% explain themselves in twenty words or more. Both criteria were fixed before running the count, and both are means over a wide spread: the six busiest projects range from 40% to 95%. A commit message is the one place where writing the explanation is required before you can move on. What that count establishes is corrective narrative, not learning as such: a twenty-word body explains a fix, and whether it teaches

anyone anything later is a separate question. In my own practice the log was read for history, for blame and for diffs, and almost never consulted as a store of lessons. That last part is an operator's report, not a measurement of the corpus.

Against those 477 commits stand 28 memory files containing at least one recorded cost or error. The two figures are **not unit-equivalent**, and I want to be careful here rather than quotable: one error can produce several commits, one file can hold several lessons, and the fix heuristic certainly catches some refactors. What the difference shows is that corrective work appears far more densely in commit history than in the memory layer. It does not license a clean ratio of errors to lessons, and an earlier draft of this article stated one.

The highest-signal artefact is the rarest. Reading all 50 handovers, 42 have a traps section, and about four in ten trap items describe a real error with its cause. But only two files quote a correction I made in conversation. One of those corrections is a single word, written down the day the agent described a client's development system as somewhere it would operate. That one word carries more context than the paragraph beside it. The instruction to capture such corrections was already in the handover procedure. It produced 2 files out of 50. Writing the same instruction again, louder, was never going to fix that.

Lessons without a home get copied, and copies drift. One lesson, confirm which system a connector points at before you call it, was born from a day of work redone after a query hit the wrong client's system. I found it in 15 files across three layers. One memory file existed under the same name in three project folders: two copies identical, the third already diverged. That is how you end up with two versions of a rule competing, and no way to tell which one is current.

The same measurement showed which format holds up. My global instruction file holds thirteen rules, six of them carrying a dated incident and a reason: the highest observed density of incident-anchored rules of any artefact in the corpus. The 56 per-project instruction files carry four narratives between them. Density is what was measured. Whether that density makes a rule more likely to be followed, or an error less likely to repeat, was not. And the two handovers richest in lessons had grown to 1,044 and 787 lines, because growing was the only way those lessons survived each rewrite.

The hypothesis that fell

After two rounds of measurement the conclusion looked clean. Every lesson that existed in memory shared one feature: I had seen the damage. The ones that looked lost were different. A missing

`await` across fifty calls. A database returning numbers as strings. A non-breaking hyphen inside a fiscal identifier that nobody would ever be able to search for. All three were found and fixed by the agent alone, in the same session, with nobody watching.

The hypothesis wrote itself. An error only becomes a lesson when someone witnesses it. Error that reaches the human turns into conversation, conversation turns into a spoken correction, and the spoken correction is what triggers the memory. Error the agent fixes on its own dies in the commit.

It even explained three numbers that had looked unrelated. The 71% of fix commits with a narrative: the agent does write the lesson, in the one place where writing is mandatory. The 2 handovers out of 50: written at the end of the day, when the error has gone cold. The 17% of memories with a cost: written when I reacted, and no reaction meant no trigger.

It was a satisfying finding. It was also wrong.

The third round was an adversarial review with one rule: judge the claims against the artefact and do not accept the author's reasoning. It demanded verification of the four lost lessons. Two of them were written, complete, in a memory file, with the mechanism, the cause of invisibility and a generalised rule:

BUG PATTERN #1 — missing `await`: *a Promise is truthy, so `getSetting(x) || []` returned the Promise, and `JSON.stringify(Promise)` gives `"{}"`. **Lesson:** every getter now **REQUIRES** `await`.*

Nobody witnessed that error. The agent found it, fixed it and wrote it up alone. The witness hypothesis was a tendency, not a law, and the counter-examples had been sitting in the data the whole time.

So why did the analysis miss them? Because the agent searched for "invisible character" when the lesson said U+2011. Because it searched docs/ when the lesson lived in the memory tree. Because it searched in the words it would use today for something written in the words of the day it happened. An audit about memory retrieval failed at memory retrieval, for precisely the reason it was about to explain. I think that is the most useful thing three nights of work produced, and it is worth more than the hypothesis it replaced.

Part of the original claim survives. Loss is real, but it is partial and driven by vocabulary, not total and driven by the absence of a witness. One of the four lessons, the U+2011 case, which is the most mechanical and most reusable of them, genuinely exists only in a commit. Another lives in a per-project decision log where no other project can see it. What does not survive is the law. And what

changes the design is the corollary, stated as precisely as the evidence allows: in this investigation, retrieval proved to be at least as large a bottleneck as capture, and it was the one the analysis had missed. Whether it is the dominant bottleneck across the whole corpus is a claim the recall log described later is meant to settle, and has not settled yet. The corpus was already writing its lessons. Any design that produced more text would have made things worse.

What survived

Four principles. Each has the evidence behind it, and where one exists, the independent source that arrived at the same place by a different route.

1. Keep the fix, never the failure

The error is the teacher, not the content. You keep what you learned. A lesson records the correction, stated positively. The error itself enters as one line of real cost and otherwise stays in `git log`, which is already an archive of errors with dates and diffs.

The corpus points this way, without ranking anything. Negative rules with no incident behind them were the ones most visibly vulnerable to ageing: 21 of 167 memories were a rule with no case attached, and one rule in my global file stayed false for weeks until somebody checked it against the code. No systematic comparison of ageing across memory types was made, so read this as the clearest cases rather than as a ranking. Independently, the open-source agent Hermes (Nous Research, MIT) writes the same exclusion into the prompt that governs what its background reviewer may keep, for the same reason. Negative claims, it says, "harden into refusals the agent cites against itself for months after the actual problem was fixed". Two sources that never met, one measuring a corpus and one designing an agent, landing on the same rule.

2. Retrieval is a bottleneck, and it was the one we had missed

The lesson gets written in the vocabulary of whoever lived it and searched for in the vocabulary of whoever needs it. Two consequences follow. Title every lesson by its trigger, meaning the situation as it will look next time, not the name of the bug. And keep the lessons in one file the agent reads whole, so nothing depends on guessing the right search term. This is not a new observation outside AI. A 2002 review of NASA's lessons-learned system found it technically sound and organisationally unused, with "no time" and a perceived intolerance for admitting mistakes as the barriers, and recommended replacing consultation with profile-matched delivery [1]. The pattern repeats across the field: what fails is the archive that waits to be searched.

3. A budget instead of a pruning policy — the weakest of the four

Nobody prunes out of virtue. People prune because something does not fit. A hard line limit is the only rule here that enforces itself: the file refuses to grow, so the next addition forces somebody to look at what is stale or duplicated. When it is full, consolidate rather than split.

This principle needs a warning the other three do not. What the corpus demonstrates is the **diagnosis**: scattered copies drift, and search by vocabulary fails. It does not demonstrate that one whole-file read is the right answer at any scale. A single 400-line file works for one operator with a few dozen lessons and an agent that reads it cheaply. Hundreds of lessons, several agents, conflicting contexts or expensive context windows could all break it, and none of those conditions has been tested here. Treat the single file as a design hypothesis chosen for this case, not as a result. It is the claim in this article a critic should attack first.

On this one the independent source does not converge. Hermes also consolidates rather than splits, but it does so on a periodic, idle-time review, which is the policy this principle argues against. The budget trigger rests on the corpus evidence alone.

4. Deep process, short result

The investigation behind this took three rounds and an adversarial review and produced several hundred lines. Five of those lines went into the lessons file. The depth is what makes those five lines worth trusting, and the brevity is what makes them usable. A method that stores the investigation instead of the result drowns in its own text.

The design that came out of the case

What follows is a design derived from the measurements above, not a validated result. The evidence supports the diagnosis; the shape of the answer is a choice, and the parts of it that have never run are listed at the end of this section. The adoptable version is in `metodo/`, with ready-to-copy files in `templates/`. In short:

Format. Five lines per lesson. The trigger is the title. Then *When* (the cue), *Do* (the rule, stated positively), *Because* (why it was invisible), and *Cost* (the real case, with a date). Links between neighbouring lessons.

Filter. Four things never enter: environment failures the human can fix, negative claims about tools, transient errors that resolved on their own, and unresolved attempts dressed up as a recommendation.

Budget. One file, a line limit (mine is 400), consolidate when full.

Cycle. At the end of a working period the agent harvests fix commits that carry a narrative body and proposes one to three lessons, adding any corrections I made in conversation. I approve them, and nothing is written without being shown. The agent reads the file back before building anything new and before debugging, because every exemplary lesson in this corpus surfaced during debugging rather than during construction.

Before adopting any of it we ran one **pilot acceptance test**, and it is worth being precise about what it does and does not show. The assumption that would sink the method is that the agent's proposals are good enough to approve rather than rewrite. So we took the busiest day of fix commits in the window. That day contained 41 fix commits across the corpus; 30 of them came from a single dominant project, and 29 of those 30 carried a full narrative. The agent proposed lessons with no hand-tuning, and the cut-off was fixed before looking: more than 3 rejected out of 8 and the design gets abandoned. The result was 8 proposed and 8 accepted **as written, with no edits**, reviewed in a few minutes. A second run on a different domain, my creative AI-video project, again with the cut-off fixed in advance, produced 10 accepted out of 10, and the lessons were qualitatively different: prompt writing, detecting fake measurements, protocol failures. Extraction is not tied to one kind of work.

What that test shows is feasibility: the pipeline can produce candidates a human will accept. What it does not show is quality, retention or retrieval, and the circularity is real. The same ecosystem proposed the lessons and approved them, I was the only reviewer, and the day chosen was the one whose dominant project had the best commit messages in the window. A repository with poor commit messages has not been tested at all. Calling this result a validation of the method would be exactly the overselling this study is trying to avoid.

The method applied to itself

We ran the making of this article through the cycle, which is the closest thing to a live demonstration here. The adversarial critic reviewed the finished text and knocked down three claims, one of them the article's own favourite piece of convergence, and the corrections are visible in the text above. The session that wrote the article was harvested and produced three lessons: sanitising prose is not sanitising scripts; an exclusion pattern that excludes contents but not the

folder; convergence asserted from the memory of a reading rather than the reading itself. All three were approved and filed like any others. The recall log picked up its first two candidate observations before its first scheduled run: my own associative memory surfaced a creative project as lesson-rich when the index had not, and the teacher's question triggered a full harvest outside any routine.

Not every part has been exercised, and honesty requires the list. The format, the filter, the harvest, the approval step and the critic have all run for real. The budget's consolidation has not, because the file is still well under its limit, and neither has the recall log's scheduled cycle. A method for learning from errors that had not learned from its own would be advertising. This one was built to eat its own cooking, and it did.

What is observed, what is chosen, and what is untested

Observed in the corpus	Corrective work is written far more densely into commit messages than into any memory layer (93% / 71% of 477 fixes). The same lesson was found in 15 files across three layers, with one copy already diverged. Two lessons believed lost were written and complete, and the search for them failed on vocabulary and location. The artefact with the highest density of incident-anchored rules is a single global file.
Derived design choice	One file, read whole. A hard line budget with consolidation instead of splitting. Five-line format with the trigger as title. The four-item exclusion filter. Harvest from fix commits, human approval before writing.
Not yet tested	Whether the single file holds at hundreds of lessons, with several agents or expensive context. Whether the lessons are retained or retrieved when needed. Whether extraction works on a repository with poor commit messages. Whether the budget's consolidation step produces good merges. Whether the recall log detects association at all.

What is still open

Human memory is not linear and it is not consulted by decision. Soup at fifty summons your mother's kitchen at four. A smell in a restaurant summons a meeting fifteen years gone. The cue

has no logical relationship to the content. What links them is the context recorded alongside. Two consequences the method above does not solve.

An index has one door; an association has many. Indexing by trigger is an advance, but it is still one condition, chosen by whoever wrote the lesson, at the moment of writing. In a network the same lesson is reachable by paths nobody anticipated.

Nobody decides to store a memory. You do not walk into a supermarket and think, I will remember this fruit. Memory forms on its own and is later called or never called. That condemns any mechanism depending on somebody remembering to write, and it explains, better than anything else I can point to, why the highest-signal section of the handover existed in 2 files out of 50. The only mechanism we read that comes close is a reviewer running in the background, writing without being asked. That is Hermes again, and it is exactly the design this method declined to adopt, because an agent writing its own manual unsupervised was the risk I was least willing to take.

So the rule is to collect without building, and the method ships an instrument for it. One question at each harvest: did any lesson fire, and what summoned it? One appended line per event, classed as title-match, association, or miss. Months of that log will decide, on evidence, whether an associative layer deserves to exist, in this corpus and in any other that keeps the same log. Nothing phones home. Anyone who wants to share an observation has an issue template, structure only. Nothing gets built until the material is thick enough to design on evidence rather than intuition, which is how the first phase began. The `[[links]]` in every lesson are the one bridge left in place. Not the network, but its substrate.

Two smaller gaps are carried rather than absorbed. The cycle captures nothing from a conversation that produced no code and no correction. And it has no step for the near miss disguised as a success, the "where we got lucky" field that the postmortem tradition keeps precisely because an unexamined near miss reads as a win and makes people bolder next time. Neither is solved here.

Limitations

Stated plainly, because the value of this work is that it is evidence rather than opinion, and evidence has edges.

- **n = 1.** One corpus, one user, one agent. These numbers describe this practice. They are not a property of agents in general.

- **The pilot acceptance test is feasibility, not efficacy.** One project, one day, commit messages above the corpus average, one reviewer, and the same ecosystem proposing and approving. It shows the pipeline produces acceptable candidates. It does not show that the lessons are good, that they are retained, or that they are retrieved when needed.
- **No controlled comparison.** There is no arm in which the method was not applied. The claim is that the design follows from the measurements, not that it beats an alternative.
- **Classification by reading,** already flagged where the numbers appear. Three of the counts are judgements made by the agent reading every file, not measurements, and the agent is a party to the study.
- **The single-file architecture is untested at scale.** The diagnosis is evidenced; the answer is a design choice for a corpus of this size. See principle 3.
- **The literature comes from other populations.** The effect sizes cited for after-action reviews, implementation intentions and near-miss research come from teams, soldiers and organisations, not from one consultant and an agent. They motivate the design. They do not validate it.
- **The first draft claimed one convergence too many,** that Hermes consolidates on a budget. It does not. The adversarial review caught it and the correction is in the text. It stays here because a rule of this work is that whatever got knocked down stays visible.
- **The analyst is a party.** The measurements, the fallen hypothesis and the rewrite were all produced by the agent whose memory is under study. The adversarial review reduces that problem. It does not remove it.

References

The literature below motivated design choices; none of it was produced from this corpus, and the populations differ from a single operator with an agent. Fuller notes on each, including where sources disagree, are kept with the research material behind this article.

- [1] United States General Accounting Office. "NASA: Better Mechanisms Needed for Sharing Lessons Learned." Report GAO-02-195. Washington, DC, January 2002.
<https://www.gao.gov/products/gao-02-195> Cited for: the finding that a technically sound lessons-learned system went unused, with lack of time and perceived intolerance for admitting mistakes as the reported barriers, and the recommendation to replace consultation

with profile-matched delivery. *Applied here as:* the argument for delivering a lesson at the point of need rather than storing it to be searched.

- [2] Keiser, N. L., and Arthur, W., Jr. "A meta-analysis of the effectiveness of the after-action review (or debrief) and factors that influence its effectiveness." *Journal of Applied Psychology* 106, no. 7 (2021): 1007-1032. <https://doi.org/10.1037/apl0000821> *Cited for:* an overall effect of $d = 0.79$ across 61 studies, and the framing of the debrief around the gap between expected and actual outcomes. *Applied here as:* the reason a review fires on successes as well as failures. *Population:* teams in training and organisational settings, not a single operator.
- [3] Dillon, R. L., and Tinsley, C. H. "How near-misses influence decision making under risk: A missed opportunity for learning." *Management Science* 54, no. 8 (2008): 1425-1440. <https://doi.org/10.1287/mnsc.1080.0869> *Cited for:* near-misses being read as successes, and increasing subsequent risk-taking. *Applied here as:* the reason the cycle's missing "where we got lucky" step is named as a gap rather than ignored.
- [4] Dillon, R. L., Tinsley, C. H., Madsen, P. M., and Rogers, E. W. "Organizational correctives for improving recognition of near-miss events." *Journal of Management* 42, no. 3 (2016): 671-697. <https://doi.org/10.1177/0149206313498905> *Cited for:* learning from a near-miss occurring when it falls into a recognised category. *Applied here as:* support for indexing a lesson by its trigger.
- [5] Gollwitzer, P. M. "Implementation intentions: Strong effects of simple plans." *American Psychologist* 54, no. 7 (1999): 493-503. https://cancercontrol.cancer.gov/sites/default/files/2020-06/goal_intent_attain.pdf *Cited for:* the "when X, then Y" form, whose mechanism is automatic detection of the situation rather than motivation. *Applied here as:* the reason a lesson is titled by its trigger and not by the name of the bug. *Caveat:* effects reported are smaller for complex behaviours than for simple ones.
- [6] Dekker, S. *The Field Guide to Understanding Human Error*. 3rd ed. Farnham: Ashgate, 2014. *Cited for:* hindsight bias, and the degradation of one's memory of what was expected. *Applied here as:* the argument for harvesting close to the event rather than at the end of a long day.
- [7] NASA Aviation Safety Reporting System (ASRS). <https://asrs.arc.nasa.gov/> *Cited for:* de-identified reporting as the mechanism that makes error reports possible at scale. *Applied here as:* context only; this study has one operator and no anonymity problem to solve.
- [8] Nous Research. Hermes Agent. MIT licence. <https://github.com/NousResearch/hermes-agent> *Cited for:* the "Do NOT capture" block in `agent/background_review.py`, read 23 Aug 2026,

and the periodic curator in `agent/curator.py`. *Applied here as:* an independent implementation that converges on principle 1 and diverges on principle 3.

Origin

Work by **Rogério Faleiros (RFERP LLC)**, developed in conversation with a coding agent between 21 and 23 August 2026, from a working corpus of client and personal projects. The formulations *the error is the teacher*, *memory summoned by a cue* and *the dream of perfection* are mine. The measurements come from the real corpus, anonymised. The complete investigation, with the discussion behind each decision, stays in a private repository. What is publishable is here.

The independent implementation reference is [8].